

软光栅化（Software Rasterization）完整图形学学习指南

目录

- 1. 数学基础
- 2. 坐标系统与变换
- 3. 光栅化流程
- 4. Z-Buffer深度测试
- 5. 扫描线填充算法
- 6. Bresenham直线算法
- 7. 项目实现解析

数学基础

1. 向量与点（Vector & Point）

Vec2、Vec3、Vec4 的定义

在您的项目中定义了三个向量类：

```
class Vec3 {
    float x, y, z;
    // 向量加法、减法、标量乘法
};
```

为什么需要不同维度的向量？

- **Vec3**：表示3D空间中的点或方向（位置、法线、光线）
- **Vec4**：齐次坐标，用于矩阵变换（w分量为齐次坐标，通常w=1表示点，w=0表示方向）

向量运算的几何意义：

```
加法：v1 + v2 = 位移
减法：v1 - v2 = 方向向量
标量乘法：v * 2 = 沿着v方向延长2倍
```

2. 向量点积（Dot Product）

```
float dot(const Vec3 &a, const Vec3 &b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
```

几何意义： $\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos(\theta)$

应用：

- 判断两个向量的夹角关系
- 计算向量在某个方向上的投影长度
- 光照计算中的漫反射强度

3. 向量叉积 (Cross Product)

```
Vec3 cross(const Vec3 &a, const Vec3 &b) {  
    return Vec3(  
        a.y * b.z - a.z * b.y,  
        a.z * b.x - a.x * b.z,  
        a.x * b.y - a.y * b.x  
    );  
}
```

几何意义：

- 结果是一个垂直于a和b的向量
- 长度 = $\|\vec{a}\| \|\vec{b}\| \sin(\theta)$
- 方向遵循右手法则

应用：

- 计算三角形面法线
- 判断三个点的方向（顺时针或逆时针）
- LookAt矩阵的构建

4. 向量归一化 (Normalization)

```
Vec3 normalize(const Vec3 &v) {  
    float length = std::sqrt(v.x * v.x + v.y * v.y + v.z * v.z);  
    if (length != 0) {  
        return Vec3(v.x / length, v.y / length, v.z / length);  
    }  
    return v;  
}
```

作用： 将向量转换为单位长度（长度=1），用于表示方向而忽略大小。

坐标系统与变换

1. 矩阵变换 (Matrix Transformation)

为什么使用4×4矩阵？

在2D或3D图形学中，我们需要进行：

- **平移 (Translation)** → 加法运算
- **旋转 (Rotation)** → 复杂的线性组合
- **缩放 (Scaling)** → 乘法运算

使用4×4矩阵和齐次坐标，所有变换都可以统一为**矩阵乘法**：

$$\begin{bmatrix} x' \\ y' \\ z' \\ w' \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

基本变换矩阵

平移矩阵 (Translation) :

$$T(tx, ty, tz) = \begin{bmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

效果： $(x, y, z) \rightarrow (x+tx, y+ty, z+tz)$

缩放矩阵 (Scaling) :

$$S(sx, sy, sz) = \begin{bmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

效果： $(x, y, z) \rightarrow (x \cdot sx, y \cdot sy, z \cdot sz)$

旋转矩阵 (Rotation) :

绕X轴旋转角度 θ ：

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

绕Y轴旋转角度 θ :

```
Ry( $\theta$ ) =  
[cos( $\theta$ )  0  sin( $\theta$ )  0]  
[0       1  0       0]  
[-sin( $\theta$ ) 0  cos( $\theta$ )  0]  
[0       0  0       1]
```

绕Z轴旋转角度 θ :

```
Rz( $\theta$ ) =  
[cos( $\theta$ ) -sin( $\theta$ )  0  0]  
[sin( $\theta$ )  cos( $\theta$ )  0  0]  
[0       0       1  0]  
[0       0       0  1]
```

2. 模型空间到世界空间 (Model to World)

您的项目中:

```
Mat4 modelMat(const Vec3 &scale, const float &angleX, const float &angleY,  
              const float &angleZ, const Vec3 &translation)  
{  
    Mat4 S = Mat4::scaling(scale.x, scale.y, scale.z);  
    Mat4 Rx = Mat4::rotation_x(angleX);  
    Mat4 Ry = Mat4::rotation_y(angleY);  
    Mat4 Rz = Mat4::rotation_z(angleZ);  
    Mat4 T = Mat4::translation(translation.x, translation.y,  
translation.z);  
    return T * Rz * Ry * Rx * S;  
}
```

顺序很重要! $T * Rz * Ry * Rx * S$ 表示:

1. 先缩放 (S)
2. 再旋转 ($Rx \rightarrow Ry \rightarrow Rz$)
3. 最后平移 (T)

这是因为矩阵乘法从**右往左**作用于顶点。

3. 视图矩阵 (View Matrix) - LookAt

```
Mat4 lookAt(Vec3 eye, Vec3 target, Vec3 up) {  
    Vec3 z = normalize(eye - target); // -z方向 (相机看的反方向)  
    Vec3 x = normalize(cross(up, z)); // x方向  
    Vec3 y = normalize(cross(z, x)); // y方向
```

```

    // 构建旋转和平移矩阵
    Mat4 res;
    // ... 设置矩阵元素
    return res;
}

```

作用： 将世界坐标系转换到**相机坐标系**

参数含义：

- **eye**: 相机位置（例如 (0, 0, 5) 表示相机在z=5处）
- **target**: 相机看向的点（例如 (0, 0, 0) 表示看向原点）
- **up**: 相机"向上"的方向向量（通常是 (0, 1, 0)）

数学原理：

1. **z轴** = 从target指向eye的方向（相机的后方）
2. **x轴** = up叉积z的方向（相机的右方）
3. **y轴** = z叉积x的方向（相机的上方）

这三个轴构成一个新的坐标系，所有顶点都被变换到这个新坐标系。

4. 投影矩阵（Projection Matrix） - Perspective

```

Mat4 perspective(float fov, float aspect, float near, float far) {
    Mat4 res = Mat4::zero();
    float tanHalfFov = std::tan(fov / 2.0f);
    res.m[0][0] = 1.0f / (aspect * tanHalfFov);
    res.m[1][1] = 1.0f / (tanHalfFov);
    res.m[2][2] = -(far + near) / (far - near);
    res.m[2][3] = -(2.0f * far * near) / (far - near);
    res.m[3][2] = -1.0f;
    return res;
}

```

作用： 将3D视景物投影到2D屏幕（模拟透视效果）

参数含义：

- **fov**: 视场角（Field of View），单位弧度。fov越大，视角越宽
- **aspect**: 屏幕宽高比（width / height）
- **near**: 近裁剪面距离（相机前面的平面）
- **far**: 远裁剪面距离（相机后面的平面）

透视效果的原理： $x_{ndc} = \frac{x_{view}}{-z_{view}} \cdot \tan(fov/2)$

投影后，离相机越远的物体越小，形成透视效果。

5. 视口矩阵（Viewport Matrix）

```
Mat4 viewport(int x, int y, int w, int h) {  
    Mat4 res = Mat4::zero();  
    res.m[0][0] = w / 2.0f;  
    res.m[0][3] = x + w / 2.0f;  
    res.m[1][1] = -h / 2.0f; // 翻转Y轴!  
    res.m[1][3] = y + h / 2.0f;  
    res.m[2][2] = 0.5f;  
    res.m[2][3] = 0.5f;  
    res.m[3][3] = 1.0f;  
    return res;  
}
```

作用： 将NDC坐标（-1到1）转换到屏幕像素坐标（0到width/height）

注意： `res.m[1][1] = -h / 2.0f` 表示**翻转Y轴**，因为：

- 数学坐标系：Y轴向上为正
- 屏幕坐标系：Y轴向下为正（像素(0,0)在左上角）

6. 完整变换管线

```
Vec3 finalPos = Viewport * Projection * View * Model * vertexPos;
```

变换顺序：

1. **Model** → 从模型空间到世界空间
2. **View** → 从世界空间到相机空间
3. **Projection** → 从相机空间到NDC空间（透视除法）
4. **Viewport** → 从NDC空间到屏幕空间

光栅化流程

1. 什么是光栅化？

光栅化是将**矢量图形**（数学定义的形状）转换为**栅格图像**（像素）的过程。

步骤：

```
3D顶点坐标 → 投影到2D屏幕 → 确定哪些像素属于该图元 → 填充像素
```

2. 从浮点坐标到像素坐标

投影后的坐标通常是浮点数，需要转换为像素网格：

```
// 像素中心更新方案
float fx = x + 0.5f; // 像素(x,y)的中心坐标
float fy = y + 0.5f;
```

这样做的目的是在**像素中心**进行采样，而不是像素的左上角。

3. 三角形光栅化

三角形是3D图形学中最基本的图元，有以下优点：

- **平面性**：三个点总是定义一个平面
- **可分解性**：任何多边形都可以分解为三角形
- **硬件友好**：GPU天然支持三角形扫描转换

Z-Buffer深度测试

1. 深度冲突（Depth Fighting）问题

当多个三角形覆盖同一个像素时，如何判断哪个三角形在前面？

答案： 比较深度值（Z坐标）

2. Z-Buffer算法

```
std::vector<float> depth_buffer(w * h, 1e9f); // 初始化为无穷远

// 在光栅化时
int idx = y * canvas.width + x;
if (z < zbuffer[idx]) { // 新的深度值更小（更靠近相机）？
    zbuffer[idx] = z;
    canvas.set_pixel(x, y, color); // 更新像素颜色
}
```

原理：

- Z坐标越小 = 离相机越近
- 每个像素保存**最近的（最小的）Z值**
- 只有Z值更小的三角形才能覆盖之前的像素

3. 深度测试的计算

在扫描线填充时，需要对每个像素计算正确的深度值：

```
// 在扫描线上进行深度插值
float t = (fx - xl) / span;
float z = lerp(zl, zr, t); // 线性插值深度
```

这样可以保证三角形内部的深度值平滑过渡。

4. 深度可视化

项目中提供了 `visualizeDepth()` 函数，可以将深度值映射为灰度值：

```
float t = clampf((z - minZ) / range, 0.0f, 1.0f);
unsigned char c = static_cast<unsigned char>(lerp(255.0f, 0.0f, t));
```

- 白色 (255) 表示距离相机最远 (minZ)
- 黑色 (0) 表示距离相机最近 (maxZ)

扫描线填充算法

1. 基本思想

扫描线填充是从上到下遍历三角形所有水平扫描线，对每条扫描线找出与三角形的交点，然后填充交点之间的像素。

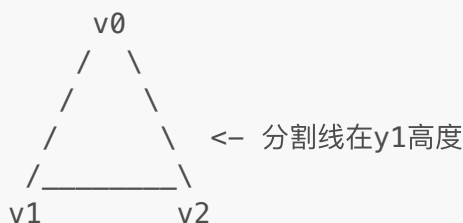
2. 三角形顶点排序

```
if (v1.y < v0.y) std::swap(v0, v1);
if (v2.y < v0.y) std::swap(v0, v2);
if (v2.y < v1.y) std::swap(v1, v2);
```

排序后的顺序： $y_0 \leq y_1 \leq y_2$

3. 三角分割

扫描线遍历分为两个阶段：



上半部分 (y0到y1)：

- 左边界：v0-v1
- 右边界：v0-v2

下半部分 (y1到y2)：

- 左边界：v1-v2

- 右边界: v0-v2

4. 扫描线与边的交点计算

使用**线性插值**计算扫描线与三边的交点:

```
auto edge = [](const Vec3 &a, const Vec3 &b, float y, float &x, float &z)
{
    float dy = b.y - a.y;
    float t = (std::abs(dy) < 1e-6f) ? 0.0f : (y - a.y) / dy;
    t = clampf(t, 0.0f, 1.0f);
    x = lerp(a.x, b.x, t);      // X坐标插值
    z = lerp(a.z, b.z, t);      // Z坐标插值 (用于深度测试)
};
```

在扫描线y处, 计算与边的交点X坐标和Z坐标。

5. 像素填充

对每条扫描线, 在左右交点之间填充像素:

```
for (int x = minX; x <= maxX; ++x) {
    float fx = x + 0.5f;
    float t = (fx - xl) / span; // 在扫描线上的参数t
    float z = lerp(zl, zr, t);  // 深度插值

    int idx = y * canvas.width + x;
    if (z < zbuffer[idx]) {      // 深度测试
        zbuffer[idx] = z;
        canvas.set_pixel(x, y, color);
    }
}
```

6. 重要的细节

为什么使用 $y + 0.5f$ 和 $x + 0.5f$?

- 在像素的**中心**进行采样
- 避免边界精度问题
- 更精确的光栅化结果

为什么需要边界检查?

```
minY = std::max(minY, 0);
maxY = std::min(maxY, canvas.height - 1);
minX = std::max(minX, 0);
maxX = std::min(maxX, canvas.width - 1);
```

防止越界访问像素或深度缓冲。

Bresenham直线算法

1. 为什么需要直线光栅化？

直线是两个端点之间的数学连线，但屏幕是离散的像素网格，需要找到最接近直线的像素序列。

2. 算法原理

Bresenham算法使用**增量计算**和**整数运算**，避免浮点除法。

```
void draw_line(Canvas &canvas, int x0, int y0, int x1, int y1, const Color
&color) {
    int dx = std::abs(x1 - x0);
    int dy = std::abs(y1 - y0);
    int sx = (x0 < x1) ? 1 : -1;    // X的步进方向
    int sy = (y0 < y1) ? 1 : -1;    // Y的步进方向
    int err = dx - dy;               // 误差项

    while (true) {
        canvas.set_pixel(x0, y0, color);
        if (x0 == x1 && y0 == y1) break;

        err2 = err * 2;
        if (err2 > -dy) {
            err -= dy;
            x0 += sx;
        }
        if (err2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}
```

3. 误差项的含义

$err = dx - dy$ 是判断下一个像素应该在哪个方向的决策变量：

- 如果 $err > 0$ ：X方向推进比Y方向多，应该增加X
- 如果 $err < 0$ ：Y方向推进比X方向多，应该增加Y
- 如果 $err = 0$ ：X和Y同时推进（45°直线）

4. 对称性处理

算法处理**8个方向**的直线：

- 4个轴向：水平、竖直
- 4个斜向：45°、135°、225°、315°

通过 **sx** 和 **sy** 的符号组合处理所有方向。

5. 优势与局限

优势：

- 只用整数运算，速度快
- 输出像素连通，无间隙
- 逻辑简单，易于硬件实现

局限：

- 不支持反走样（Anti-aliasing）
- 不同方向的直线质量可能不同

项目实施解析

1. 项目结构

软光栅/

main.cpp

main

op_*.tga

完整的软光栅实现

编译后的可执行文件

输出的图像文件

2. 主要类和函数

类/函数	功能	重要性
Vec2, Vec3, Vec4	向量类	★★★★★
Mat4	4×4矩阵类，处理所有变换	★★★★★
Canvas	画布，像素缓冲	★★★★★
Color	颜色（RGBA）	★★★
Triangle	三角形类	★★★★★
draw_line()	Bresenham直线算法	★★★★★
rasterizeTriangleScanline()	扫描线填充+Z-Buffer	★★★★★
lookAt()	视图矩阵	★★★★★
perspective()	投影矩阵	★★★★★
viewport()	视口矩阵	★★★
save_as_tga()	输出TGA格式图像	★★★

3. 执行流程

```
int main() {
    // 1. 创建画布和深度缓冲
    Canvas canvas(800, 600);
    std::vector<float> depth_buffer(800 * 600, 1e9f);

    // 2. 定义立方体顶点和三角形索引
    std::vector<Vec3> cubeVerts = {...};
    std::vector<std::array<int, 3>> cubeTris = {...};

    // 3. 构建变换矩阵
    Mat4 m = modelMat(...); // 模型矩阵
    Mat4 view = lookAt(...); // 视图矩阵
    Mat4 projection = perspective(...); // 投影矩阵
    Mat4 vp = viewport(...); // 视口矩阵

    // 4. 变换所有顶点
    for (const auto &v : cubeVerts) {
        clipVerts.push_back(vp * projection * view * m * v);
    }

    // 5. 光栅化每个三角形
    for (const auto &tri : cubeTris) {
        Vec3 a = clipVerts[tri[0]];
        Vec3 b = clipVerts[tri[1]];
        Vec3 c = clipVerts[tri[2]];
        rasterizeTriangleScanline(canvas, depth_buffer, a, b, c, color);
    }

    // 6. 保存结果
    save_as_tga("output.tga", 800, 600, canvas.pixels);
}
```

4. 立方体渲染示例

立方体有**12个三角形**（6个面，每个面2个三角形）：

```
std::vector<std::array<int, 3>> cubeTris = {
    {0, 1, 2}, {0, 2, 3}, // 后面 (back)
    {4, 6, 5}, {4, 7, 6}, // 前面 (front)
    {0, 4, 5}, {0, 5, 1}, // 底面 (bottom)
    {3, 2, 6}, {3, 6, 7}, // 顶面 (top)
    {1, 5, 6}, {1, 6, 2}, // 右侧 (right)
    {0, 3, 7}, {0, 7, 4} // 左侧 (left)
};
```

每个面使用不同的颜色，通过Z-Buffer确保正确的遮挡关系。

5. 命令行参数

```
./main --wire          # 只显示线框
./main --no-fill       # 不填充三角形
./main --depth         # 显示深度可视化
./main --fill --wire   # 既显示填充又显示线框
```

6. 关键的数值设置

```
// 透视投影参数
float fov = 3.14159f / 4.0f;          // 45° 视场角
float aspect = 800.0f / 600.0f;      // 宽高比
float near = 0.1f;                   // 近裁剪面
float far = 100.0f;                  // 远裁剪面

// 相机设置
Vec3 eye = Vec3(0, 0, 5);            // 相机位置 (沿Z轴)
Vec3 target = Vec3(0, 0, 0);         // 看向原点
Vec3 up = Vec3(0, 1, 0);             // 向上方向

// 模型变换
Vec3 scale = Vec3(1, 1, 1);          // 不缩放
float angleX = 0.6f;                 // 绕X轴旋转
float angleY = 0.8f;                 // 绕Y轴旋转
float angleZ = 0.0f;                 // 不绕Z轴旋转
Vec3 translation = Vec3(0, 0, 0);    // 不平移
```

学习重点总结

核心概念（必须掌握）

1. 向量运算：点积、叉积、归一化

- 点积用于判断角度关系和光照
- 叉积用于计算法线和方向

2. 矩阵变换：平移、旋转、缩放、透视投影

- 理解每个矩阵的几何意义
- 掌握矩阵乘法的顺序

3. 坐标系统转换

- 模型空间 → 世界空间 → 视图空间 → NDC空间 → 屏幕空间
- 理解齐次坐标和透视除法

4. 光栅化算法

- 三角形扫描线填充
- Z-Buffer深度测试

- Bresenham直线算法

实践技能

1. 调试技巧

- 使用深度可视化诊断问题
- 分步打印矩阵值
- 线框模式检查顶点位置

2. 优化方向

- 整数运算替代浮点
- 缓冲预分配
- SIMD向量化

3. 扩展应能

- 添加纹理映射（UV坐标插值）
- 实现Gouraud着色（顶点颜色插值）
- Phong光照模型
- 法线映射（Normal Mapping）

参考资料

推荐书籍

- 《计算机图形学》（Foley & van Dam）
- 《Real-Time Rendering》
- 《Shader X》系列

在线资源

- LearnOpenGL.com
- ScratchAPixel
- Lighthouse3D.com

进阶主题

- 四元数旋转（Quaternion）
- 裁剪平面（Clipping）
- 光栅化的反走样（MSAA）
- 延迟渲染（Deferred Rendering）

学习建议：

1. 先理解数学基础，特别是矩阵和向量
2. 手动跟踪一个简单三角形的完整变换过程
3. 逐步修改参数观察效果变化

4. 编写可视化调试工具（如深度图、法线图）
5. 尝试添加新功能（如纹理、光照）